



DAYANANDA SAGAR
UNIVERSITY

Devarakaggalahalli, Harohalli, Kanakapura Road,

Dt. Ramanagara, Karnataka -562112

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(DATA SCIENCE)

Project Title: Student Management System

Subject: Skill Enhancement Course (SEC) – OOPs With Java

Submitted By:

Name: Sshyamolika Gungun

Roll No.: 60

Class/Semester: 3rd sem

Submitted To:

Teacher's Name: Mriganka sir

Academic Year: 2025–2026

2. Abstract

This project titled “**Student Management System**” is developed as part of the Skill Enhancement Course (SEC) on **OOPs with Java**. The main objective of this project is to implement core Object-Oriented Programming concepts through a simple but functional application.

The system stores student information such as roll number, name, and course. It uses Java classes, objects, constructors, and file handling to design a program that can add student records, display them, and save them into a file. The project demonstrates the practical use of:

- Encapsulation
- Method overriding
- Data structures (ArrayList)
- Java I/O operations
- Class and object modeling

This project serves as a foundation for building more advanced management systems.

3. Introduction

Java is a powerful, platform-independent, object-oriented programming language widely used for application development. The Skill Enhancement Course (SEC) on OOPs with Java requires students to understand and apply the fundamentals of object-oriented concepts.

This **Student Management System** is created to demonstrate:

- How OOP principles can be used in real programs
 - How data can be stored dynamically
- How text files can be used as a simple data storage mechanism

The program is console-based, making it easy to understand and suitable for beginner-level OOP implementation.

4. Objectives of the Project

The major objectives of this project are:

- 1.To develop a basic student management system using Java
2. To apply Object-Oriented Programming concepts practically
3. To use classes, objects, constructors, and methods
4. To implement dynamic data storage using ArrayList
5. To use Java File Handling (FileWriter) to save data
6. To generate structured output showing stored student data
7. To create a neat, functional mini-project for SEC OOP with Java.

5. Project Scope

The scope of this project includes:

- Adding multiple students
- Displaying all stored student records
- Writing the student records to a text file
- Demonstrating OOP concepts clearly

Not included but possible as future scope:

- Searching records
- Updating and deleting records
- GUI using Swing/JavaFX
- Database connectivity using JDBC
- Login system

6. Methodology

The methodology used in this project is based on object-oriented design.

A. Class Design

1. Student Class

Represents a student with:

- rollNo
- name
- course

Includes:

- Constructor for initialization
- toString() to convert object data into readable format

2. StudentManagementSystem Class

Handles:

- Adding students
- Displaying students
- Saving students to file

Uses:

- Static ArrayList<Student> as storage
- FileWriter to store details in students.txt

7. System Requirements

Software Requirements

- Windows/Linux/Mac OS
- JDK version 8 or higher
- Any Java editor (Notepad, VS Code, IntelliJ IDEA, Eclipse)

Hardware Requirements

- 2 GB RAM
- Basic processor
- 1 MB free disk space for program and output file

8. Program Code

```
import java.util.*;

import java.io.*;

class Student {

    int rollNo;

    String name;

    String course;

    Student(int r, String n, String c) {

        rollNo = r;

        name = n;

        course = c;

    }

    public String toString() {

        return rollNo + " " + name + " " + course;

    }

}

class StudentManagementSystem {

    static ArrayList<Student> students = new ArrayList<>();

    static void addStudent(int roll, String name, String course) {

        students.add(new Student(roll, name, course));

        System.out.println("Student added successfully.");

    }

}
```

```
static void displayStudents() {  
    for (Student s : students) {  
        System.out.println(s);  
    }  
}
```

```
static void saveToFile() {  
    try {  
        FileWriter fw = new FileWriter("students.txt");  
        for (Student s : students) {  
            fw.write(s + "\n");  
        }  
        fw.close();  
        System.out.println("Data saved to file.");  
    } catch (IOException e) {  
        System.out.println("Error: " + e.getMessage());  
    }  
}
```

```
public static void main(String[] args) {  
    addStudent(1, "John", "CSE");  
    addStudent(2, "Alice", "ECE");  
    addStudent(3, "Bob", "MECH");  
  
    displayStudents();  
    saveToFile();  
}
```

9. Explanation of Code

1. Student Class

- Stores the basic attributes of a student
- Constructor initializes the roll number, name, and course
- toString() returns the record in string format

2. StudentManagementSystem Class

- Contains an ArrayList to store student objects
- addStudent() creates and adds student entries
- displayStudents() prints all student records
- saveToFile() exports the data to a text file

3. Main Method

- Adds three sample students
- Displays them
- Saves them into students.txt

10. Output

Console Output

Student added successfully.

Student added successfully.

Student added successfully.

1 John CSE

2 Alice ECE

3 Bob MECH

Data saved to file.

File Output (students.txt)

1 John CSE

2 Alice ECE

3 Bob MECH

11. Conclusion

This project provides a practical understanding of OOP principles through a simple but effective Student Management System. It demonstrates how Java is used to design classes, manage data, and perform file operations. The system successfully stores, displays, and writes student information to a file.

This mini project helps strengthen Java fundamentals and serves as a foundation for advanced applications involving GUI or databases.

12. References

- Oracle Java Documentation
- OOPs with Java Class Notes
- Online Java Tutorials (GeeksForGeeks, W3Schools)
- Faculty Notes